

Limited Time-Series Extension

Purpose

The purpose of this section is to assess whether the aggregate annual EFN service series supports a limited time-series forecasting exercise and, if so, to construct a conservative baseline projection under transparent assumptions.

This analysis focuses on total service encounters aggregated by year. Given the short annual sample and the pronounced instability observed after 2020, the emphasis is not on developing a high-confidence forecasting model, but on evaluating what can be learned from a simple univariate time-series approach and documenting the resulting uncertainty.

Accordingly, this section is best understood as a descriptive time-series extension to the broader service-demand analysis rather than as a standalone forecasting project. All results are intended to provide exploratory insight and a cautious planning baseline, not causal inference, policy evaluation, or precise prediction.

Scope and Structure

This section proceeds as follows:

1. Construction of the aggregate annual service series
2. Time-series diagnostics and stationarity assessment
3. Parsimonious model comparison and selection
4. Residual diagnostics and adequacy checks
5. Baseline forecast generation and interpretation

The goal is not to develop a high-complexity forecasting system, but to assess what a simple univariate time-series approach can reasonably support given the short annual sample and observed post-2020 instability.

Notes on Interpretation

Results in this section should be interpreted as a conservative baseline conditional on historical patterns in the observed series. They do not incorporate exogenous shocks, policy changes, funding shifts, or structural changes beyond

those reflected in the historical data.

Because the analysis relies on an annual series with only 11 observations, forecast precision is limited and uncertainty should be emphasized over point estimates.

```
In [16]: # Core data handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns

# Time series diagnostics and modeling
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox

# Model evaluation
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Display settings
pd.set_option("display.float_format", "{:,.2f}".format)
plt.style.use("seaborn-v0_8-whitegrid")

df = pd.read_excel("2014-2024_meals_no_partners.xlsx")
```

1. Construction of the Aggregate Annual Service Series

This section constructs the aggregate annual series of total service encounters used in the time-series extension. Consistent with the preceding descriptive analysis, the primary series is total service encounters, defined as the total number of recorded client visits across all service units within a given year.

Several aggregate outcome variables are available in the dataset, including:

- Total service encounters (clients served)
- Total unduplicated clients
- Total food distributed (pounds)

Total service encounters are selected as the primary series because they most directly capture aggregate service activity over time and are closely related to operational considerations such as staffing, capacity, and system load. By

contrast, food distribution volumes reflect both client demand and operational factors—including sourcing, inventory composition, and packaging decisions—which may introduce additional variation unrelated to underlying service activity. Unduplicated client counts are useful for measuring service reach but do not capture repeat utilization.

Aggregation is performed at the annual level to align with the temporal structure of the dataset and to avoid imposing artificial seasonality on a short series. The resulting annual series serves as the basis for the diagnostic and baseline forecasting exercises that follow.

All analysis in this section is descriptive and intended to provide exploratory time-series insight and a conservative planning baseline rather than causal inference, policy evaluation, or precise prediction.

Aggregate Total Service Encounters By Year

```
In [2]: ts_clients = (
        df
        .groupby("year", as_index=False)
        .agg(total_clients=("total_clients", "sum"))
        .sort_values("year")
        )

ts_clients
```

```
Out[2]:
```

	year	total_clients
0	2014	1309195
1	2015	1414290
2	2016	1377194
3	2017	1360974
4	2018	1335373
5	2019	1375411
6	2020	1822792
7	2021	1423014
8	2022	2073753
9	2023	2688188
10	2024	3154795

Convert to a time-series indexed by year

```
In [3]: # Convert to a time-series indexed by year (year-end timestamps)
ts_clients_series = (
    ts_clients
    .set_index("year")["total_clients"]
    .sort_index()
)

# Build a proper annual DateTimeIndex: 2014 -> 2014-12-31, etc.
ts_clients_series.index = pd.to_datetime(
    ts_clients_series.index.astype(int).astype(str) + "-12-31"
)

# Explicitly set annual frequency (year-end)
ts_clients_series = ts_clients_series.asfreq("YE-DEC")

ts_clients_series
```

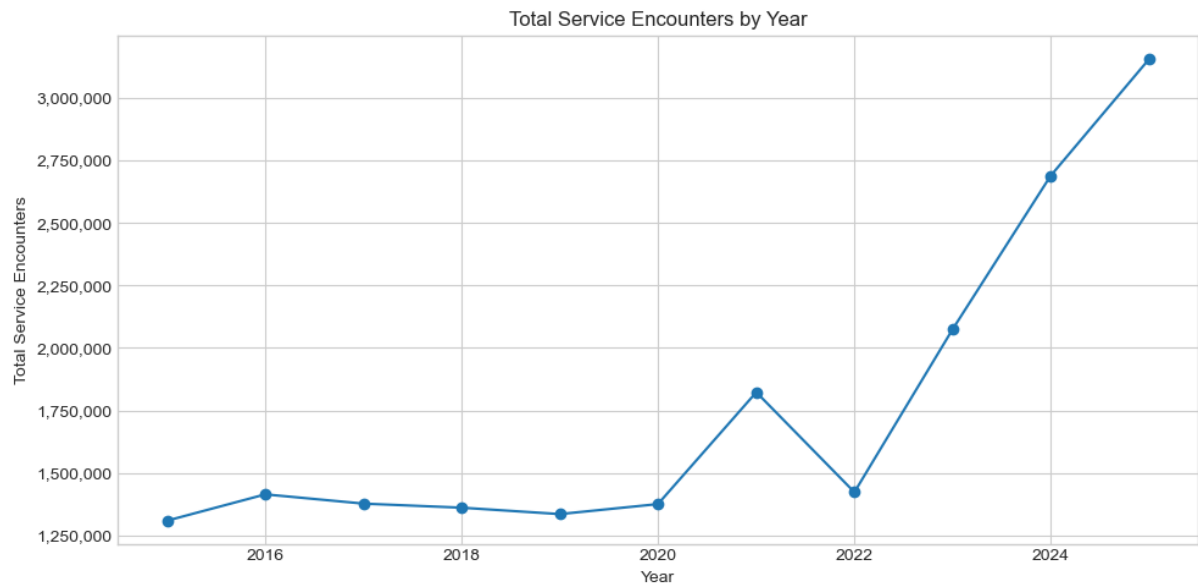
```
Out[3]: year
2014-12-31    1309195
2015-12-31    1414290
2016-12-31    1377194
2017-12-31    1360974
2018-12-31    1335373
2019-12-31    1375411
2020-12-31    1822792
2021-12-31    1423014
2022-12-31    2073753
2023-12-31    2688188
2024-12-31    3154795
Freq: YE-DEC, Name: total_clients, dtype: int64
```

```
In [4]: plt.figure(figsize=(10, 5))
plt.plot(ts_clients_series, marker="o")

plt.title("Total Service Encounters by Year")
plt.xlabel("Year")
plt.ylabel("Total Service Encounters")

plt.ticklabel_format(style="plain", axis="y")
plt.gca().yaxis.set_major_formatter(
    mtick.StrMethodFormatter("{x:,.0f}")
)

plt.grid(True)
plt.tight_layout()
plt.show()
```



The annual series of total service encounters is relatively stable in the pre-2020 period, with only modest year-to-year variation. A pronounced increase occurs in 2020, followed by a sharp decline in 2021. After 2021, the series rises strongly through 2024, reaching the highest levels observed in the sample.

This pattern represents a clear departure from the relatively stable pre-2020 behavior, with increased volatility and nonlinear movement in the post-2019 period. These visual features justify a cautious diagnostic assessment of whether the series supports even a limited univariate forecasting exercise.

2. Time-Series Diagnostics and Stationarity Assessment

This section evaluates the time-series properties of the aggregate annual service series, with particular attention to stationarity. Because the series is short, annual, and visibly unstable after 2020, the goal is not to impose a complex model structure, but to assess whether a simple univariate time-series extension is methodologically defensible.

Stationarity is examined using two complementary statistical tests:

- The Augmented Dickey–Fuller (ADF) test, which tests the null hypothesis of a unit root.
- The Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test, which tests the null hypothesis of stationarity.

Using both tests jointly provides a more robust assessment of the series' stochastic properties and helps guide transformation and parsimonious specification choices in the sections that follow.

Augmented Dickey-Fuller (ADF) Test

```
In [5]: adf_result = adfuller(ts_clients_series)

adf_output = pd.Series(
    {
        "ADF Statistic": adf_result[0],
        "p-value": adf_result[1],
        "Used Lags": adf_result[2],
        "Number of Observations": adf_result[3]
    }
)

adf_output
```

```
Out[5]: ADF Statistic      3.20
p-value      1.00
Used Lags    2.00
Number of Observations  8.00
dtype: float64
```

Kwiatkowski-Phillips-Schmidt-Shin (KPSS)

```
In [6]: kpss_stat, kpss_pvalue, kpss_lags, kpss_crit = kpss(
    ts_clients_series,
    regression="c",
    nlags="auto"
)

kpss_output = pd.Series(
    {
        "KPSS Statistic": kpss_stat,
        "p-value": kpss_pvalue,
        "Used Lags": kpss_lags
    }
)

kpss_output
```

```
Out[6]: KPSS Statistic    0.50
p-value      0.04
Used Lags    1.00
dtype: float64
```

The stationarity diagnostics provide consistent evidence that the aggregate annual series of total service encounters is nonstationary in levels.

The Augmented Dickey-Fuller (ADF) test fails to reject the null hypothesis of a unit root (p-value = 1.00), indicating the presence of nonstationary behavior. Complementarily, the KPSS test rejects the null hypothesis of stationarity at

conventional significance levels (p -value = 0.04), further supporting the conclusion that the series is not stationary around a constant mean.

Taken together, these results suggest that the series exhibits a stochastic trend and requires transformation before further time-series analysis. In particular, first differencing is considered to assess whether the series can be rendered approximately stationary for a limited univariate extension. Transformation choices are evaluated in the sections that follow.

First Differencing of the Aggregate Service Encounters Series

```
In [7]: ts_clients_diff = ts_clients_series.diff().dropna()

ts_clients_diff
```

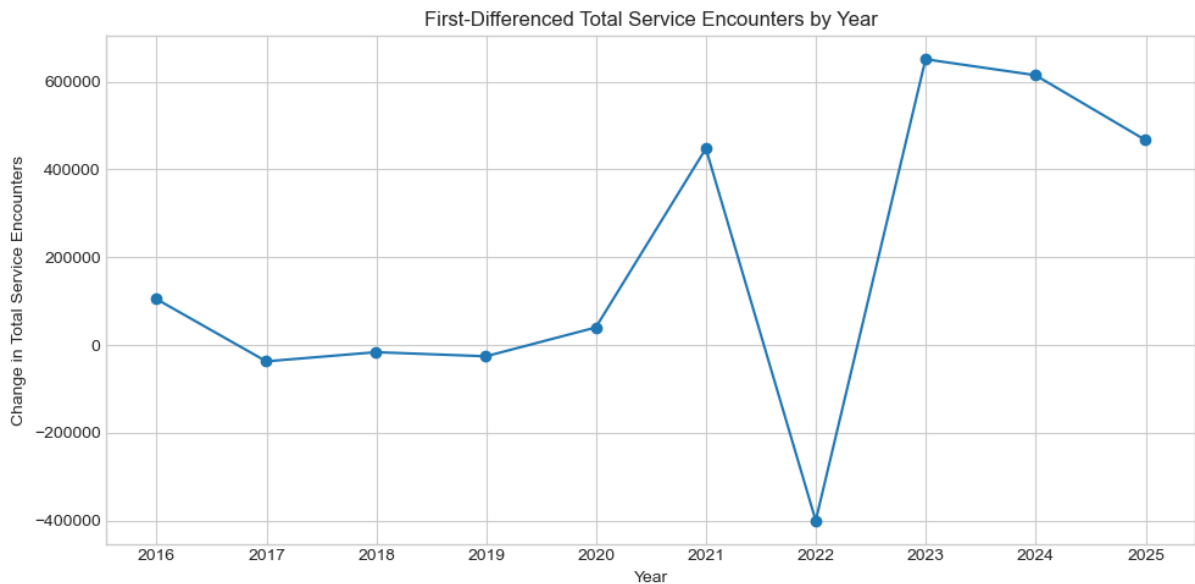
```
Out[7]: year
2015-12-31    105,095.00
2016-12-31    -37,096.00
2017-12-31    -16,220.00
2018-12-31    -25,601.00
2019-12-31     40,038.00
2020-12-31    447,381.00
2021-12-31   -399,778.00
2022-12-31    650,739.00
2023-12-31    614,435.00
2024-12-31    466,607.00
Freq: YE-DEC, Name: total_clients, dtype: float64
```

Visual Inspection of Differenced Series

```
In [8]: plt.figure(figsize=(10, 5))
plt.plot(ts_clients_diff, marker="o")

plt.title("First-Differenced Total Service Encounters by Year")
plt.xlabel("Year")
plt.ylabel("Change in Total Service Encounters")

plt.ticklabel_format(style="plain", axis="y")
plt.grid(True)
plt.tight_layout()
plt.show()
```



The first-differenced series displays substantial volatility, with relatively small year-to-year changes prior to 2020 and large positive and negative movements thereafter. In particular, the pronounced increase in 2020, followed by a sharp decline in 2021 and strong rebounds in subsequent years, highlights the presence of large transitory shocks rather than smooth, persistent growth.

While the differenced series appears more centered around zero than the level series, visual inspection alone is insufficient to assess stationarity. Accordingly, formal unit-root and stationarity tests are conducted in the next subsection to evaluate whether differencing adequately stabilizes the series.

ADF Test (Differenced Series)

```
In [9]: adf_result_diff = adfuller(ts_clients_diff)

adf_output_diff = pd.Series(
    {
        "ADF Statistic": adf_result_diff[0],
        "p-value": adf_result_diff[1],
        "Used Lags": adf_result_diff[2],
        "Number of Observations": adf_result_diff[3],
    }
)

adf_output_diff
```

```
Out[9]: ADF Statistic      -2.55
p-value                   0.10
Used Lags                  0.00
Number of Observations    9.00
dtype: float64
```


KPSS Test (Differenced Series)

```
In [10]: kpss_stat_diff, kpss_pvalue_diff, kpss_lags_diff, _ = kpss(
        ts_clients_diff,
        regression="c",
        nlags="auto"
    )

kpss_output_diff = pd.Series(
    {
        "KPSS Statistic": kpss_stat_diff,
        "p-value": kpss_pvalue_diff,
        "Used Lags": kpss_lags_diff,
    }
)

kpss_output_diff
```

```
Out[10]: KPSS Statistic    0.37
p-value                0.09
Used Lags              0.00
dtype: float64
```

Results from unit root testing suggest that the first-differenced series is approximately stationary. The Augmented Dickey–Fuller (ADF) test does not reject the null hypothesis of a unit root at conventional significance levels ($p = 0.10$), reflecting limited test power given the short sample length and the presence of large pandemic-era shocks. In contrast, the KPSS test fails to reject the null hypothesis of stationarity ($p = 0.09$). Taken together, these results are consistent with a difference-stationary process.

While inference is necessarily limited by the small annual sample, the combined ADF and KPSS evidence suggests that first differencing substantially reduces the nonstationary behavior observed in the level series. First differencing is therefore adopted for subsequent model specification.

Given this evidence, further insight into the dynamic structure of the differenced series is obtained by examining its autocorrelation and partial autocorrelation functions (ACF and PACF). These diagnostics help characterize any remaining serial dependence and indicate whether a highly parsimonious specification is more appropriate than a richer dynamic model.

ACF and PACF of Differenced Series

```
In [11]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

        plot_acf(ts_clients_diff, ax=axes[0], lags=4)
```

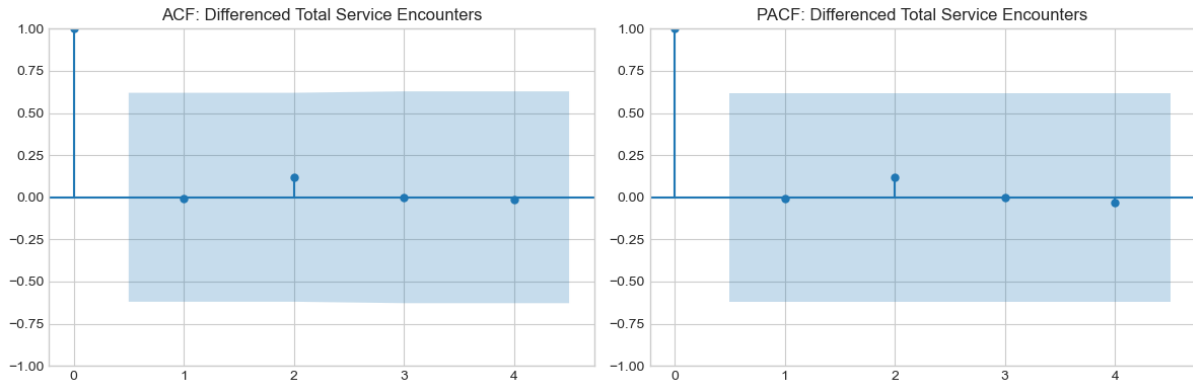
```

axes[0].set_title("ACF: Differenced Total Service Encounters")

plot_pacf(ts_clients_diff, ax=axes[1], lags=4, method="yw")
axes[1].set_title("PACF: Differenced Total Service Encounters")

plt.tight_layout()
plt.show()

```



The autocorrelation function (ACF) and partial autocorrelation function (PACF) of the differenced series exhibit no statistically significant spikes at lags 1–4. This indicates that, after differencing, the series displays minimal remaining serial dependence and behavior consistent with a weakly stationary process.

The absence of pronounced autoregressive (AR) or moving-average (MA) structure in the low-order lags suggests that highly parsimonious specifications are appropriate. Given the short annual sample and limited power of correlation diagnostics, model selection focuses on simple ARIMA structures that avoid overparameterization while remaining consistent with the observed dependence patterns.

3. Parsimonious Model Comparison and Baseline Specification

This section compares a small set of parsimonious ARIMA specifications that are consistent with the preceding diagnostics and with the limited annual sample size.

Given the short length of the series and the absence of strong low-order autocorrelation in the differenced data, the objective is not to search over a large model space, but to evaluate whether a simple univariate specification can provide a reasonable baseline representation of the observed series.

The candidate models considered are:

- **ARIMA(0,1,0)** — random walk baseline

- **ARIMA(0,1,1)** — moving-average component in first differences
- **ARIMA(1,1,0)** — autoregressive component in first differences

Candidate ARIMA Specifications

```
In [12]: models = {
    "ARIMA(0,1,0)": SARIMAX(ts_clients_series, order=(0, 1, 0), trend=
    "ARIMA(0,1,1)": SARIMAX(ts_clients_series, order=(0, 1, 1), trend=
    "ARIMA(1,1,0)": SARIMAX(ts_clients_series, order=(1, 1, 0), trend=
}

results = {}

for name, model in models.items():
    results[name] = model.fit(dispatch=False)

# Compare candidate models (information criteria)
model_compare = pd.DataFrame(
    {
        "AIC": {k: v.aic for k, v in results.items()},
        "BIC": {k: v.bic for k, v in results.items()},
    }
).sort_values("AIC")

model_compare
```

```
Out[12]:
```

	AIC	BIC
ARIMA(0,1,0)	287.03	287.33
ARIMA(1,1,0)	288.62	289.23
ARIMA(0,1,1)	289.80	290.40

Candidate ARIMA models are compared using the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC), which balance in-sample fit against model complexity.

Across the candidate specifications, **ARIMA(0,1,0)** yields the lowest AIC and BIC values, indicating the most favorable trade-off between fit and parsimony within this limited comparison set. Specifications with additional autoregressive or moving-average terms do not improve fit enough to justify their added complexity.

This result is consistent with the earlier diagnostics. Unit root tests suggest nonstationarity in levels but approximate stationarity after first differencing, while the ACF and PACF of the differenced series show no strong residual serial dependence. Taken together, these findings support a highly parsimonious

baseline specification.

Accordingly, **ARIMA(0,1,0)** is retained as the baseline model for the exploratory forecasting exercise that follows.

4. Residual Diagnostics and Adequacy Checks

This section evaluates whether the selected baseline specification provides an adequate statistical representation of the observed annual service series.

In a well-specified ARIMA model, residuals should behave approximately like white noise: they should fluctuate around zero, exhibit no clear remaining serial dependence, and avoid strong systematic patterns over time. If these conditions are not satisfied, even a simple baseline projection may be misleading.

Given the limited length of the annual series, model adequacy is assessed using a small, targeted set of diagnostics chosen to balance statistical rigor with interpretability in a small-sample setting.

Diagnostics Employed:

- **Ljung–Box test:** a formal portmanteau test of the null hypothesis that residuals are independently distributed up to a specified lag. Failure to reject the null is interpreted as an absence of detected residual autocorrelation rather than definitive proof of independence.
- **Residual time-series plot:** used to visually assess whether residuals fluctuate around zero and to identify potential outliers, structural shifts, or periods of unusual variance.

Autocorrelation and partial autocorrelation analysis was conducted earlier during model identification using the differenced series. Given the absence of strong low-order dependence at that stage and the highly parsimonious selected specification, additional residual ACF analysis is not emphasized here.

While the SARIMAX estimation output reports additional diagnostics, including normality and heteroskedasticity tests, these are interpreted cautiously because their reliability is limited in such a short sample. The diagnostics emphasized here are used to assess the adequacy of the baseline specification before proceeding to the forecast extension.

All diagnostics are interpreted descriptively and are used solely to assess whether the baseline specification is adequate for a limited forecasting extension.

Ljung-Box Test

```
In [13]: # Fit the selected model (based on lowest AIC/BIC among candidates)
best_model = SARIMAX(ts_clients_series, order=(0, 1, 0), trend="n")
best_result = best_model.fit(dispatch=False)

# Residuals (drop NaNs that can occur at the start due to differencing)
resid = best_result.resid.dropna()

best_result.summary()
```

Out[13]:

SARIMAX Results						
Dep. Variable:		total_clients		No. Observations:		11
Model:		SARIMAX(0, 1, 0)		Log Likelihood		-142.515
Date:		Mon, 06 Apr 2026		AIC		287.029
Time:		15:37:49		BIC		287.332
Sample:		12-31-2014		HQIC		286.697
		- 12-31-2024				
Covariance Type:		opg				
	coef	std err	z	P> z	[0.025	0.975]
sigma2	1.267e+11	6.52e+10	1.943	0.052	-1.11e+09	2.54e+11
Ljung-Box (L1) (Q):		0.00	Jarque-Bera (JB):		0.47	
Prob(Q):		0.98	Prob(JB):		0.79	
Heteroskedasticity (H):		80.31	Skew:		-0.03	
Prob(H) (two-sided):		0.00	Kurtosis:		1.94	

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Interpretation

The Ljung-Box test at lag 1 fails to reject the null hypothesis of no residual autocorrelation (p-value = 0.98), indicating no detected remaining serial dependence at this lag. Given the small annual sample, this result should be interpreted cautiously and viewed as an absence of detected autocorrelation rather than definitive proof of independence.

The Jarque–Bera test fails to reject the null hypothesis of normally distributed residuals (p-value = 0.79). Residual skewness is near zero and kurtosis is modest, suggesting no severe departures from normality are evident, although the power of normality tests is limited in small samples.

In contrast, the heteroskedasticity test rejects the null hypothesis of constant variance (p-value < 0.01), suggesting time-varying residual variance. This result is consistent with the pronounced changes in scale observed after 2020 and indicates that uncertainty may vary over time rather than remain constant.

Overall, these diagnostics suggest that the ARIMA(0,1,0) specification is adequate as a parsimonious baseline representation of the series, while variance instability remains an important limitation. This supports proceeding to a cautious forecast extension with uncertainty emphasized over point precision.

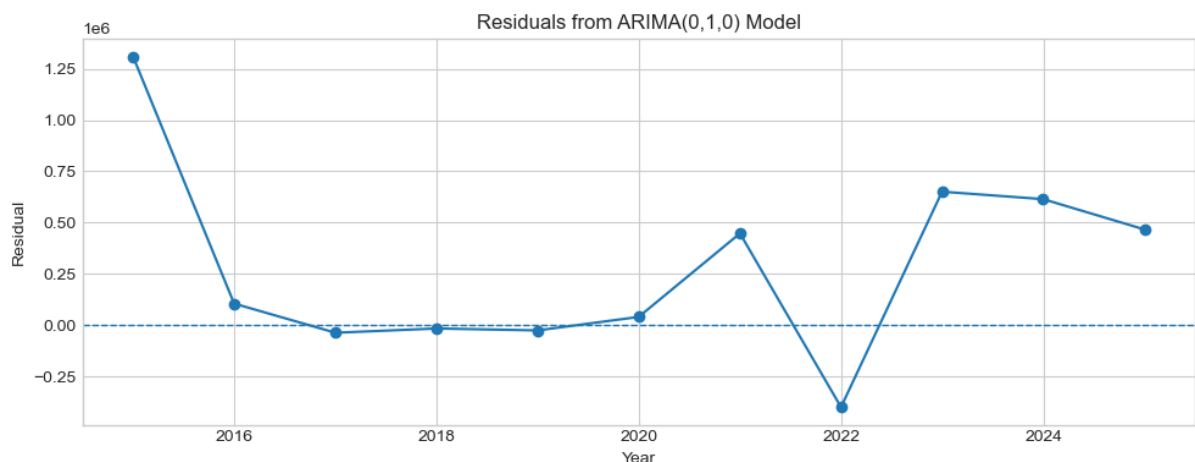
Extract and Plot Residuals

```
In [14]: # Extract and plot residuals
resid = best_result.resid.dropna()

plt.figure(figsize=(10, 4))
plt.plot(resid, marker="o")
plt.axhline(0, linestyle="--", linewidth=1)

plt.title("Residuals from ARIMA(0,1,0) Model")
plt.xlabel("Year")
plt.ylabel("Residual")

plt.grid(True)
plt.tight_layout()
plt.show()
```



Interpretation

The residual time-series plot shows that residuals fluctuate around zero with no

persistent trend or systematic pattern, indicating that the ARIMA(0,1,0) specification captures the main mean dynamics of the series. There is no clear visual evidence of remaining serial dependence, which is consistent with the Ljung–Box result and earlier autocorrelation diagnostics.

However, residual magnitude varies substantially over time, with notably larger deviations in the post-2020 period. This pattern suggests time-varying volatility rather than constant variance and reinforces the need to interpret the forecast extension conservatively.

5. Baseline Forecast Extension and Interpretation

This section uses the selected ARIMA(0,1,0) specification to generate a short-horizon baseline projection of aggregate annual service encounters.

Projections are produced at the annual frequency to align with the temporal structure of the data. Given the short sample and the structural disruption observed around 2020–2021, the horizon is intentionally limited to avoid over-extrapolation.

Both point projections and prediction intervals are reported. These intervals reflect uncertainty implied by the fitted univariate model and should be interpreted as conditional on the model assumptions and the historical series.

```
In [15]: # Generate forecasts
forecast_horizon = 3 # short-horizon forecast (years)

forecast_result = best_result.get_forecast(steps=forecast_horizon)

forecast_mean = forecast_result.predicted_mean
forecast_ci = forecast_result.conf_int()

# Create forecast index
forecast_years = pd.date_range(
    start=ts_clients_series.index[-1] + pd.offsets.YearEnd(1),
    periods=forecast_horizon,
    freq="YE-DEC"
)

forecast_mean.index = forecast_years
forecast_ci.index = forecast_years

# Plot observed series and forecasts
plt.figure(figsize=(10, 5))

plt.plot(
    ts_clients_series,
```

```

        label="Observed",
        marker="o"
    )

    plt.plot(
        forecast_mean,
        label="Forecast",
        marker="o",
        linestyle="--"
    )

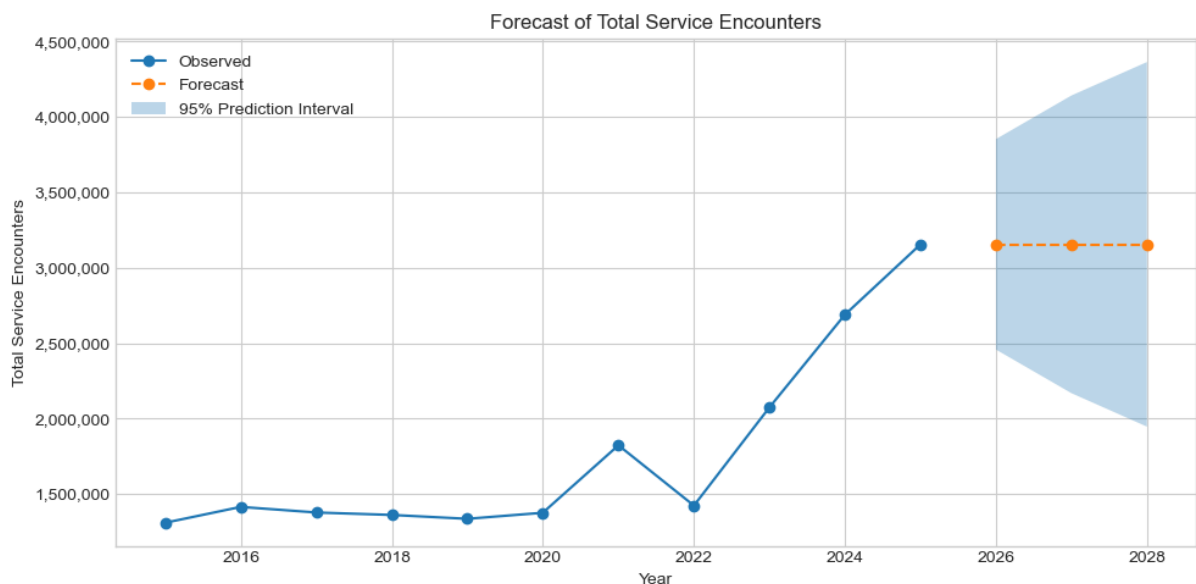
    plt.fill_between(
        forecast_ci.index,
        forecast_ci.iloc[:, 0],
        forecast_ci.iloc[:, 1],
        alpha=0.3,
        label="95% Prediction Interval"
    )

    plt.title("Forecast of Total Service Encounters")
    plt.xlabel("Year")
    plt.ylabel("Total Service Encounters")

    plt.ticklabel_format(style="plain", axis="y")
    plt.gca().yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))

    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

```



The fitted ARIMA(0,1,0) model produces flat point projections over the three-year horizon. This result follows directly from the structure of the model: after first differencing, the series is represented by a random-walk-type baseline, so the

best statistical projection of future values is the most recently observed level rather than an extrapolated growth path.

The visual gap between 2024 and the beginning of the projection reflects the fact that forecast values start in the first unobserved period after the historical sample ends. The projected values are shown separately from realized outcomes to distinguish baseline projections from recorded service activity.

In practical terms, the flat projection should not be interpreted as evidence that service demand is expected to remain unchanged. Rather, it indicates that the historical data do not provide a stable and recurring pattern strong enough to justify projecting sustained increases or decreases with confidence.

Following a period of relative stability prior to 2020, the series exhibits substantial disruption and volatility, including a pronounced increase in 2020, a decline in 2021, and strong growth thereafter. As a result, prediction intervals widen quickly over the projection horizon, highlighting considerable uncertainty around future service activity.

This imprecision is driven primarily by the short length of the annual series and the structural instability observed in the post-2020 period rather than by any claim that the baseline model is unusable. With limited observations and no clear repeating dynamics, the model appropriately reflects uncertainty instead of imposing a strong trend unsupported by the data.

Accordingly, these results are best interpreted as a conservative baseline conditional on historical behavior. They are intended to support planning under uncertainty and exploratory time-series insight rather than precise or deterministic prediction.

Limitations and Extensions

The baseline forecasts presented in this section are subject to several important methodological limitations. Most notably, the analysis relies on a short annual time series (11 observations), which constrains model complexity and limits the ability to identify stable long-run dynamics. In addition, the presence of pronounced structural volatility beginning in 2020 reduces the reliability of trend extrapolation and contributes to wide prediction intervals. Finally, the univariate modeling approach does not incorporate exogenous drivers—such as policy changes, funding levels, or macroeconomic conditions—that may influence service demand.

In addition to these modeling constraints, the interpretation of forecasts is also

shaped by how service activity is measured. Total service encounters reflect visits recorded by partner agencies and primarily capture food distribution activity, which may be supported by EFN-supplied food as well as food sourced independently by partner organizations.

These limitations suggest several natural extensions. Future work could explore alternative forecasting targets (e.g., food pounds distributed or capacity measures), incorporate exogenous regressors in a multivariate or ARIMAX framework, or model sub-annual data if higher-frequency observations become available. Disaggregated forecasting at the service-unit level may also improve planning relevance, though such approaches would require additional data and careful treatment of sparsity.